

Natural Language Processing (CS24311) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & LANGUAGE MODELS BANK

Natural Language Processing (CS24311)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024-25 Scheme)

 Complete Course Structure & NLP Architecture Matrix

Module	Core Syllabus Scope	Key Algorithms & Formulations
Module I	Morphology & Language Modeling	Regular Expressions, Porter Stemmer, Byte-Pair Encoding (BPE), N-grams, Perplexity, Laplace, Good-Turing & Kneser-Ney Smoothing
Module II	Part-of-Speech Tagging & HMM	Penn Treebank Tagset, Hidden Markov Models (HMM), Forward Algorithm, Viterbi Dynamic Programming Decoding
Module III	Syntactic Parsing & Grammars	Context-Free Grammars (CFG), Chomsky Normal Form (CNF), CYK Parsing Algorithm, Treebanks & Probabilistic CFG (PCFG)
Module IV	Semantics & Word Embeddings	WordNet Synsets, TF-IDF Vectorization, Pointwise Mutual Information (PMI), Word2Vec (Skip-Gram & CBOW), GloVe
Module V	Transformers, LLMs & Generation	Scaled Dot-Product Attention, Multi-Head Attention, Positional Encodings, BERT (Masked LM), GPT, BLEU Score & ROUGE

 **Exam Preparation & High-Yield Strategy**

This publication-grade master book consolidates all 5 modules with formal mathematical derivations, KaTeX-rendered language model smoothing formulas, Viterbi trellis dynamic programming traces, and model answers to BIT Mesra end-semester examination questions.

Module I: Morphology, Tokenization & Statistical Language Modeling

1. NLP Pipeline Hierarchy (Phonetics, Morphology, Syntax, Semantics, Pragmatics)
2. Regular Expressions & Finite-State Transducers (FST) for Morphological Parsing
3. Tokenization Challenges, Minimum Edit Distance (Levenshtein) Dynamic Programming
4. Subword Tokenization Algorithms: Byte-Pair Encoding (BPE), WordPiece & Unigram
5. Word Stemming (Porter Stemmer Rules) vs. Morphological Lemmatization
6. Statistical Language Modeling & The Chain Rule of Probability
7. N -gram Language Models (Unigram, Bigram, Trigram) & Markov Assumptions
8. Language Model Evaluation: Perplexity (PPL) Mathematical Formulation
9. Smoothing Techniques: Laplace (Add-1), Add- k , Lidstone & Absolute Discounting
10. Advanced Smoothing: Good-Turing Frequency Estimation ($r^* = (r + 1) \frac{N_r + 1}{N_r}$)
11. Modified Kneser-Ney Smoothing (Interpolation & Continuation Probability P_{cont})
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 1 & 2: Natural Language Hierarchy & Morphology

Natural Language Processing (NLP) is a multidisciplinary field at the intersection of Computer Science, Artificial Intelligence, and Computational Linguistics concerned with the automated understanding, generation, and processing of human language.

Linguistic Level	Core Linguistic Phenomenon	Key Computational Tasks
1. Phonetics & Phonology	Physical sounds and linguistic acoustic patterns.	Speech-to-Text (ASR), Text-to-Speech (TTS).
2. Morphology	Structure and formation of words from root morphemes.	Stemming, Lemmatization, Tokenization, BPE.
3. Syntax	Grammatical rules governing sentence and phrase structure.	POS Tagging, Context-Free Parsing, Dependency Parsing.
4. Semantics	Literal meaning of words, phrases, and compositions.	WordNet, Word2Vec, BERT, Semantic Role Labeling.
5. Pragmatics & Discourse	Contextual meaning, intent, coreference, and conversational tone.	Anaphora Resolution, Dialogue Systems, Sarcasm Detection.

Topic 3 & 4: Subword Tokenization (Byte-Pair Encoding - BPE)

The Byte-Pair Encoding (BPE) Algorithm (Sennrich et al., 2016)

1. Initialize vocabulary with all individual characters in the training corpus plus an end-of-word symbol ``.
2. Count the frequency of all adjacent symbol pairs in the tokenized corpus.
3. Identify the most frequent symbol pair (c_1, c_2) and merge it into a single new vocabulary token $c_1 c_2$.
4. Repeat steps 2 and 3 for a fixed number of merge operations k (e.g., 32,000 merges in GPT-2).
5. *Significance:* Solves the Out-of-Vocabulary (OOV) problem by representing rare and unseen words as sequences of subword units (e.g., `unaffordability` → `un` + `afford` + `ability`).

Topic 6 & 7: N -gram Language Modeling & The Chain Rule

1. Exact Chain Rule of Probability for Sentence $W = (w_1, w_2, \dots, w_n)$:

$$P(w_1, w_2, \dots, w_n) = \prod_{k=1}^n P(w_k \mid w_1, w_2, \dots, w_{k-1})$$

2. Bigram Language Model (1st-Order Markov Assumption):

$$P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k \mid w_{k-1}) \quad \text{where } P(w_k \mid w_{k-1}) = \frac{C(w_{k-1}, w_k)}{C(w_{k-1})}$$

3. Language Model Perplexity (PPL):

$$\text{PPL}(W) = P(w_1, w_2, \dots, w_n)^{-\frac{1}{n}} = \sqrt[n]{\frac{1}{P(w_1, w_2, \dots, w_n)}} = 2^{-\frac{1}{n} \sum_{i=1}^n \log_2 P(w_i \mid w_{i-1})}$$

Lower Perplexity indicates a superior language model that is less surprised by the test corpus!

Topic 9 – 11: Smoothing Techniques for Statistical Language Models

Smoothing Method	Probability Estimation Formula	Key Mechanics & Intuition
1. Laplace (Add-1)	$P_{\text{Laplace}}(w_i w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$	Adds 1 to every count; shifts far too much probability mass to unseen N -grams in large vocabularies V .
2. Add- k (Lidstone)	$P_{\text{Add-}k}(w_i w_{i-1}) = \frac{C(w_{i-1}, w_i) + k}{C(w_{i-1}) + kV} \quad (0 < k < 1)$	Reduces probability mass distortion compared to Add-1.
3. Absolute Discounting	$P(w_i w_{i-1}) = \frac{\max(C(w_{i-1}, w_i) - d, 0)}{C(w_{i-1})} + \frac{d \cdot \{w : C(w_{i-1}, w) > 0\} }{C(w_{i-1})} P(w_i)$	Subtracts a fixed discount $d \in (0, 1)$ from non-zero counts and redistributes mass to unigrams.
4. Kneser-Ney (Interpolated)	$P_{\text{KN}}(w_i w_{i-1}) = \frac{\max(C(w_{i-1}, w_i) - d, 0)}{C(w_{i-1})} + \lambda(w_{i-1}) P_{\text{continuation}}(w_i)$ $P_{\text{continuation}}(w_i) = \frac{ \{w' : C(w', w_i) > 0\} }{\sum_w \{w' : C(w', w) > 0\} }$	Considers how versatile word w_i is as a continuation across diverse preceding contexts (e.g., 'San Francisco'). State-of-the-art N -gram smoothing.

Top BIT Mesra Exam Questions & Answers (Module I)

Q1. Define Perplexity. Given a test sentence with probability 10^{-6} and length 6 words, compute its Perplexity. (6 Marks)

Perplexity (PPL): The reciprocal geometric mean of the probability assigned to the test sentence by the language model:

$$\text{PPL}(W) = (P(W))^{-1/N} = (10^{-6})^{-1/6} = 10^1 = \mathbf{10}$$

A perplexity of 10 means the model is as confused as if it had to choose uniformly among 10 words at each step.

Module II: Part-of-Speech Tagging & Hidden Markov Models (HMM)

1. Linguistic Word Classes: Open Class vs. Closed Class Lexical Categories
2. Penn Treebank 45-Tag Tagset (NN, NNP, VB, VBD, JJ, RB, IN, DT)
3. Part-of-Speech (POS) Tagging Approaches (Rule-Based, Statistical, Neural)
4. Hidden Markov Models (HMM) 5-Tuple: States, Observations, Transitions, Emissions, Priors
5. The 3 Fundamental Computational Problems in HMMs (Evaluation, Decoding, Learning)
6. The Forward Algorithm for Sequence Likelihood ($O(N^2T)$ Dynamic Programming)
7. The Viterbi Algorithm for Optimal Hidden State Decoding (max Trellis Trace)
8. Complete Step-by-Step Viterbi Trellis Trace on Sentence "Janet will back the bill"
9. Baum-Welch (Expectation-Maximization) Unsupervised Learning Algorithm
10. Maximum Entropy Markov Models (MEMM) & The Label Bias Problem
11. Linear-Chain Conditional Random Fields (CRF) Globally Normalized Potentials
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 1 – 4: Part-of-Speech Tagging & HMM Mathematical Architecture

Part-of-Speech (POS) Tagging is the task of assigning a grammatical category label (e.g., Noun, Verb, Adjective, Preposition) to each word in an input text sequence:

$$\mathbf{w} = (w_1, w_2, \dots, w_T) \xrightarrow{\text{POS Tagging}} \mathbf{t} = (t_1, t_2, \dots, t_T)$$

Hidden Markov Model 5-Tuple $\lambda = (S, V, A, B, \pi)$:

1. $S = \{s_1, s_2, \dots, s_N\}$: Finite set of hidden states (e.g., POS tags).
2. $V = \{v_1, v_2, \dots, v_M\}$: Finite set of observable vocabulary symbols (words).
3. $A = \{a_{ij}\}$: State Transition Probability Matrix, where $a_{ij} = P(q_t = s_j \mid q_{t-1} = s_i)$.
4. $B = \{b_j(k)\}$: Emission Probability Matrix, where $b_j(k) = P(o_t = v_k \mid q_t = s_j)$.
5. $\pi = \{\pi_i\}$: Initial State Probability Distribution, where $\pi_i = P(q_1 = s_i)$.

Topic 7 & 8: The Viterbi Dynamic Programming Decoding Algorithm

Viterbi Recurrence Relations: Let $v_t(j)$ be the highest probability of any state sequence ending in state j at time t :

Initialization ($t = 1$):

$$v_1(j) = \pi_j \cdot b_j(o_1), \quad \text{backpointer}_1(j) = 0 \quad (1 \leq j \leq N)$$

2. Recursion ($t = 2 \dots T$):

$$v_t(j) = \max_{i=1}^N \left(v_{t-1}(i) \cdot a_{ij} \right) \cdot b_j(o_t) \quad (1 \leq j \leq N)$$

$$\text{backpointer}_t(j) = \arg \max_{i=1}^N \left(v_{t-1}(i) \cdot a_{ij} \right)$$

3. Termination:

$$P^* = \max_{i=1}^N v_T(i), \quad q_T^* = \arg \max_{i=1}^N v_T(i)$$

4. Backtracking:

$$q_t^* = \text{backpointer}_{t+1}(q_{t+1}^*) \quad \text{for } t = T-1, T-2, \dots, 1$$

Topic 10 & 11: MEMMs vs. Linear-Chain CRFs (Label Bias Problem)

Model Class	Mathematical Formulation	Normalization & Key Theoretical Properties
1. Generative HMM	$P(\mathbf{w}, \mathbf{t}) = \prod_{i=1}^T P(t_i t_{i-1}) P(w_i t_i)$	Models joint probability; cannot easily integrate overlapping, non-independent features.
2. Discriminative MEMM	$P(\mathbf{t} \mathbf{w}) = \prod_{i=1}^T \frac{\exp(\mathbf{w}^T \mathbf{f}(t_i, t_{i-1}, \mathbf{w}, i))}{\sum_{t'} \exp(\mathbf{w}^T \mathbf{f}(t', t_{i-1}, \mathbf{w}, i))}$	Local Normalization at each step: Suffers from the <i>Label Bias Problem</i> where states with low-entropy transitions dominate regardless of input observations.
3. Linear-Chain CRF	$P(\mathbf{t} \mathbf{w}) = \frac{1}{Z(\mathbf{w})} \exp \left(\sum_{i=1}^T \sum_k w_k f_k(t_i, t_{i-1}, \mathbf{w}, i) \right)$	Global Normalization ($Z(\mathbf{w})$): Overcomes the Label Bias Problem completely by evaluating the total score of the entire sequence simultaneously.

Top BIT Mesra Exam Questions & Answers (Module II)

Q1. State the 3 fundamental problems of Hidden Markov Models and name the algorithms used to solve them. (6 Marks)

- Evaluation Problem (Likelihood):** Given model λ and observation sequence O , compute $P(O | \lambda)$. Solved by the **Forward Algorithm** (or Backward Algorithm) in $O(N^2T)$ time.
- Decoding Problem:** Given model λ and observation sequence O , find the optimal hidden state sequence Q^* that best explains O . Solved by the **Viterbi Algorithm** in $O(N^2T)$ time.
- Learning Problem (Parameter Estimation):** Given observation sequence O , adjust model parameters $\lambda = (A, B, \pi)$ to maximize $P(O | \lambda)$. Solved by the **Baum-Welch (EM) Algorithm**.

Module III: Syntactic Parsing, Context-Free Grammars & Dependency Structures

1. Phrase Structure Grammars & Constituency vs. Dependency Syntax Trees
2. Context-Free Grammars (CFG) for Natural Language Syntax ($G = (V, T, P, S)$)
3. Structural Ambiguity in Natural Language (Prepositional Phrase & Coordination)
4. Chomsky Normal Form (CNF) Formal Conversion Rules ($A \rightarrow BC \mid a$)
5. The CYK (Cocke-Younger-Kasami) Dynamic Programming Parsing Algorithm
6. Complete Step-by-Step CYK Triangular Table Parsing Trace for "Book the flight"
7. Probabilistic Context-Free Grammars (PCFG) & Maximum Likelihood Estimation
8. The Probabilistic CYK (Inside-Outside) Parsing Algorithm for Disambiguation
9. Dependency Grammar Foundations: Heads, Dependents & Universal Dependencies (UD)
10. Transition-Based Dependency Parsing (Shift-Reduce, ARC-STANDARD & ARC-EAGER)
11. Graph-Based Dependency Parsing (Chu-Liu-Edmonds Maximum Spanning Tree)
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 4 & 5: Chomsky Normal Form (CNF) & The CYK Algorithm

A Context-Free Grammar is in **Chomsky Normal Form (CNF)** if every production rule has one of two forms:

$$A \rightarrow BC \quad \text{or} \quad A \rightarrow a$$

Where A, B, C are non-terminals ($B, C \neq S$) and a is a single terminal symbol.

CYK Dynamic Programming Recurrence: Let table entry $P[i, j, A]$ be true if non-terminal A derives substring $w_i \dots w_j$ of length $l = j - i + 1$. **Base Case (Length $l = 1$):**

$$P[i, i, A] = \text{true} \quad \text{for all } A \rightarrow w_i \in P$$

2. Inductive Step (Length $l = 2 \dots n$):

$$P[i, j, A] = \text{true} \iff \exists k \in [i, j-1] \text{ and } A \rightarrow BC \in P \text{ such that } P[i, k, B] = \text{true} \wedge P[k+1, j, C] = \text{true}$$

Time Complexity: $O(n^3 \cdot |P|)$ where n is sentence length and $|P|$ is number of CNF grammar rules.

Topic 9 & 10: Dependency Parsing & Transition Systems

Dependency Syntax describes grammatical structure by establishing direct binary asymmetric grammatical relations between words (Heads $\rightarrow$ Dependents), without intermediate phrase nodes:

Transition Action	Stack & Buffer Manipulation	Dependency Arc Created
1. SHIFT	Remove first word w from Buffer and push onto Stack.	None.
2. LEFT-ARC (r)	Creates dependency arc $s_1 \xleftarrow{r} s_2$ from top of stack s_1 to second item s_2 . Pops s_2 from stack.	$s_1 \rightarrow s_2$ (s_1 is Head).
3. RIGHT-ARC (r)	Creates dependency arc $s_1 \xrightarrow{r} s_2$ from second item s_1 to top of stack s_2 . Pops s_2 from stack.	$s_1 \rightarrow s_2$ (s_1 is Head).

Q1. State the Chomsky Normal Form (CNF) restrictions and explain why CNF is mandatory for the CYK algorithm. (8 Marks)

- **CNF Restrictions:** All productions must strictly be of the form $A \rightarrow BC$ (exactly 2 non-terminals) or $A \rightarrow a$ (exactly 1 terminal). No unit productions ($A \rightarrow B$) or ϵ -productions ($A \rightarrow \epsilon$) are allowed.
- **Why Mandatory for CYK:** CYK relies on dynamic programming by splitting a substring into exactly two parts ($w[i \dots k]$ and $w[k + 1 \dots j]$). Binary branching ($A \rightarrow BC$) guarantees that any subphrase is formed by joining exactly two adjacent subphrases, creating the $O(n^3)$ tabular recurrence.

Module IV: Lexical Semantics, Vector Space Models & Distributed Word Embeddings

1. Lexical Semantics: Word Senses, Synonyms, Antonyms, Hypernyms & Hyponyms
2. WordNet Ontology Architecture: Synsets, Taxonomic Distance & Similarity Metrics
3. Distributional Hypothesis: "You shall know a word by the company it keeps"
4. Term-Document Matrix & Term Frequency-Inverse Document Frequency (TF-IDF)
5. Term-Context Matrices & Pointwise Mutual Information (PMI) / Positive PMI
6. Dimensionality Reduction in Semantics: Latent Semantic Analysis (LSA / SVD)
7. Word2Vec Architecture (Mikolov et al., 2013): Continuous Bag-of-Words (CBOW)
8. Word2Vec Skip-Gram Model with Negative Sampling (SGNS) Mathematical Objective
9. Vector Arithmetic & Analogical Reasoning ($\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} + \mathbf{v}_{\text{Woman}} \approx \mathbf{v}_{\text{Queen}}$)
10. GloVe (Global Vectors for Word Representation): Log-Bilinear Matrix Factorization
11. FastText: Subword N-Gram Embeddings for Morphology & Out-of-Vocabulary (OOV)
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 4 & 5: Vector Space Models (TF-IDF & Positive PMI)

1. Term Frequency-Inverse Document Frequency (TF-IDF):

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$
$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}, \quad \text{IDF}(t, D) = \log_{10} \left(\frac{|D|}{|\{d \in D : t \in d\}|} \right)$$

2. Positive Pointwise Mutual Information (PPMI):

$$\text{PMI}(w, c) = \log_2 \left(\frac{P(w, c)}{P(w)P(c)} \right) = \log_2 \left(\frac{C(w, c) \cdot N}{C(w) \cdot C(c)} \right)$$
$$\text{PPMI}(w, c) = \max(\text{PMI}(w, c), 0)$$

Topic 7 & 8: Word2Vec (Skip-Gram with Negative Sampling – SGNS)

Word2Vec learns dense low-dimensional continuous vector embeddings ($d \approx 300$) where semantically similar words are mapped to proximate points in vector space.

Skip-Gram with Negative Sampling (SGNS) Loss Function

For a target word w with true context word c_{pos} and k randomly drawn negative words $c_{\text{neg},1} \dots c_{\text{neg},k}$ from noise distribution $P_n(w) \propto f(w)^{3/4}$:

$$\mathcal{L}_{\text{SGNS}} = \log \sigma(\mathbf{v}_{c_{\text{pos}}}^T \mathbf{v}_w) + \sum_{i=1}^k \mathbb{E}_{c_{\text{neg},i} \sim P_n} \left[\log \sigma(-\mathbf{v}_{c_{\text{neg},i}}^T \mathbf{v}_w) \right]$$

Where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function. Maximizes dot product with actual context words while pushing away negative noise samples!

Topic 10: GloVe (Global Vectors for Word Representation)

GloVe Log-Bilinear Objective Function (Pennington, Socher, Manning, 2014):

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(\mathbf{w}_i^T \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

Where X_{ij} is the co-occurrence count of word i with context word j , and $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a weighting function that caps the influence of extremely frequent stop words (with $\alpha = 0.75, x_{\max} = 100$).

Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare CBOW and Skip-Gram models in Word2Vec across 4 architectural parameters. (8 Marks)

1. **Prediction Task:** CBOW predicts the target center word given a window of surrounding context words; Skip-Gram predicts the surrounding context words given the center word.
2. **Training Speed:** CBOW trains several times faster than Skip-Gram because context word vectors are averaged in a single forward pass.
3. **Performance on Rare Words:** Skip-Gram is substantially superior for rare words and small datasets because each target-context pair is treated as a separate training sample.
4. **Vector Representations:** CBOW smooths over context distribution; Skip-Gram preserves multi-sense representations.

Module V: Transformers, Large Language Models & Generation Evaluation

1. Limitations of Recurrent Architectures (RNN, LSTM, GRU) & Sequential Bottleneck
2. The Transformer Architecture (Vaswani et al., 2017: "Attention Is All You Need")
3. Scaled Dot-Product Self-Attention Mechanism & Softmax Normalization ($\sqrt{d_k}$)
4. Multi-Head Attention Formulations ($\text{Concat}(\text{head}_1 \dots \text{head}_h)W^O$)
5. Positional Encoding Schemes: Fixed Sinusoidal Waves vs. Rotary Embeddings (RoPE)
6. Transformer Encoder Blocks (LayerNorm, Residual Additions & Multi-Layer Feedforward)
7. Transformer Decoder Blocks (Masked Causal Self-Attention & Cross-Attention)
8. BERT Architecture: Bidirectional Transformer Encoders (Masked LM & NSP Tasks)
9. GPT Family: Autoregressive Decoder Models, In-Context Learning & Prompting
10. Text Generation Decoding: Greedy, Beam Search, Top- k & Top- p (Nucleus) Sampling
11. NLP Evaluation Metrics: BLEU Score (with Brevity Penalty) & ROUGE-1/2/L Math
12. Comprehensive Solved BIT Mesra & GATE Exam Question Bank (8 Questions)

Topic 2 – 4: The Transformer & Scaled Dot-Product Attention

1. Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where $Q \in \mathbb{R}^{n \times d_k}$ is the Query matrix, $K \in \mathbb{R}^{m \times d_k}$ is the Key matrix, and $V \in \mathbb{R}^{m \times d_v}$ is the Value matrix.

Why Scale by $\frac{1}{\sqrt{d_k}}$? For large values of d_k , dot products grow large in magnitude, pushing the softmax function into regions with extremely small gradients. Dividing by $\sqrt{d_k}$ stabilizes gradient backpropagation!

2. Multi-Head Attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Where projection parameter matrices $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, and $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Topic 8 & 9: BERT vs. GPT Architectural Comparison

Feature	BERT (Devlin et al., 2018)	GPT (Radford et al., OpenAI)
Core Architecture	Transformer Encoder-Only (Bidirectional attention).	Transformer Decoder-Only (Causal left-to-right masked attention).
Pre-Training Objectives	1. Masked Language Model (MLM) : Predict 15% masked tokens. 2. Next Sentence Prediction (NSP) : Binary classification.	Autoregressive Language Modeling : Predict next token $P(w_t w_{1:t-1})$.
Context Flow	Full bidirectional visibility across future and past tokens.	Strictly unidirectional past context (future tokens masked out).
Primary Strength	NLU (Classification, Named Entity Recognition, Extractive QA).	NLG (Text Generation, Summarization, Code Synthesis, Chatbots).

Topic 11: Text Generation Evaluation Metrics (BLEU & ROUGE)

1. Bilingual Evaluation Understudy (BLEU) Metric:

$$\text{BLEU} = \text{BP} \times \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Where p_n is the modified n -gram precision, and BP is the **Brevity Penalty** to penalize artificially short candidate outputs:

$$\text{BP} = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases}$$

Where c is candidate length and r is reference length.

Top BIT Mesra Exam Questions & Answers (Module V)

Q1. Explain why Self-Attention overcomes the fundamental sequential bottleneck of RNNs and LSTMs. (8 Marks)

- Massive Parallelization:** RNNs compute hidden state $h_t = f(h_{t-1}, x_t)$ sequentially, requiring $O(n)$ sequential time steps that cannot be parallelized across GPU cores. Self-Attention computes all pairwise token interactions simultaneously via matrix multiplications (QK^T) in $O(1)$ sequential operations.
- Direct Long-Range Paths:** In RNNs, information must travel step-by-step through n recurrent transitions, causing vanishing gradients for distant words. Self-Attention connects any two positions in the sequence with a direct $O(1)$ path length, enabling instantaneous long-range context resolution.